

Maven CICD Examples

When we run:

mvn clean install

Maven automatically performs several steps.

1. Old build files deleted (clean phase)

What happens

Maven deletes the **previous build output folder called target/**.

Example old structure:

ecommerce-app

```
├── src
├── pom.xml
└── target
    ├── classes
    ├── test-classes
    └── ecommerce-app-1.0.jar
```

Running:

mvn clean

removes the target folder.

Why this is important in real projects

Imagine your **ecommerce app** has:

- Product module
- Order module
- Payment module

Yesterday you compiled code.

Today you changed some classes.

If old compiled files remain, you may run **old code accidentally**.

So Maven ensures:

- ✓ Fresh build
 - ✓ No leftover compiled classes
 - ✓ Avoids build errors
-

2. Code compiled (compile phase)

What happens

Maven compiles all Java files from:

src/main/java

into .class files.

Example:

src/main/java/com/shop/service/ProductService.java

After compilation:

target/classes/com/shop/service/ProductService.class

Example in ecommerce app

You may have:

ProductService.java

OrderService.java

PaymentService.java

These files are compiled using the Java compiler.

Example code:

```
public class ProductService {  
    public List<Product> getProducts() {  
        return productRepository.findAll();  
    }  
}
```

Maven compiles this automatically.

3. Tests executed (test phase)

Maven automatically runs **unit tests** written using JUnit.

Example test file:

src/test/java/com/shop/service/ProductServiceTest.java

Example test:

```
@Test  
public void testGetProducts() {  
    ProductService service = new ProductService();  
    List<Product> products = service.getProducts();  
    assertNotNull(products);  
}
```

Why this matters in a real ecommerce project

Before deployment, Maven verifies:

- Product listing works
- Order creation works
- Payment calculation works

If **any test fails**, the build stops.

Example error:

Tests run: 10, Failures: 1

BUILD FAILED

This prevents broken code from going to production.

4. JAR/WAR created (package phase)

Now Maven packages the compiled code into a **deployable file**.

Two common formats:

- **JAR** → Java applications
- **WAR** → Web applications

Example output:

target/ecommerce-app-1.0.jar

or

target/ecommerce-app-1.0.war

Real example in ecommerce app

This packaged file contains:

ecommerce-app-1.0.jar

- └─ ProductService.class
- └─ OrderService.class
- └─ PaymentService.class
- └─ libraries

Everything needed to run the application.

5. Artifact installed locally (install phase)

Maven installs the built file into the **local Maven repository**.

Location:

~/.m2/repository

Example path:

~/.m2/repository/com/shop/ecommerce-app/1.0/ecommerce-app-1.0.jar

Why this is useful

Other projects can reuse this artifact.

Example:

Suppose you have another project:

- **payment-service**

It can use the ecommerce library like this:

```
<dependency>
  <groupId>com.shop</groupId>
  <artifactId>ecommerce-app</artifactId>
  <version>1.0</version>
</dependency>
```

Maven fetches it from the local repository.

What is the use of target/myproject-1.0.jar?

This file is the **final packaged application**.

Uses in real projects:

1 Deployment

It can be deployed to servers like:

- Apache Tomcat
- WildFly
- Docker containers

Example:

```
java -jar ecommerce-app-1.0.jar
```

Starts the ecommerce application.

2 CI/CD pipelines

In tools like:

- Jenkins
- GitHub Actions

Pipeline flow:

Developer pushes code → Maven build → JAR created → Deploy to server

3 Microservices communication

In microservice architecture, each service produces a **JAR**.

Example services:

- product-service.jar
- order-service.jar
- payment-service.jar

Each runs independently.

Real Build Flow in an Ecommerce Company

Developer pushes code → CI server runs:

mvn clean install

Maven performs:

1. Delete old builds
2. Compile Java code
3. Run tests
4. Create JAR/WAR
5. Store artifact

Then pipeline deploys it to production server.

☒ In one sentence

target/ecommerce-app-1.0.jar is the **final deployable package** of your application created by **Maven after compiling, testing, and packaging the project.**